Data Management for Machine Learning - Assignment 2
End-to-End Data Management Pipeline for a Recommendation System

Name: MIDATHANA RAJESH Roll No: 2025AG05785

1. Introduction

The aim of this assignment was to build a data pipeline for a recommendation system that goes from raw data to trained and evaluated models, with every step automated. I used a movie platform as the example. Instead of downloading a public dataset I generated synthetic data with 1,500 users, 600 items and about 59,000 raw interactions (ratings from 1 to 5 over two years). I added common data problems on purpose: missing values, duplicate rows, wrong data types, ratings outside the valid range, malformed and future timestamps, user and item IDs that do not exist, and interactions that happen before the user signed up. This gave the validation step real work to do and let me measure how much data it removed.

2. Design decisions and tools

The pipeline has eight stages: ingestion, validation, preparation, transformation, feature management, training, evaluation and orchestration (Figure 1). Each stage is a separate Python module and all settings are in one config file with a fixed random seed, so a run can be repeated. Running the pipeline twice gave identical metrics.

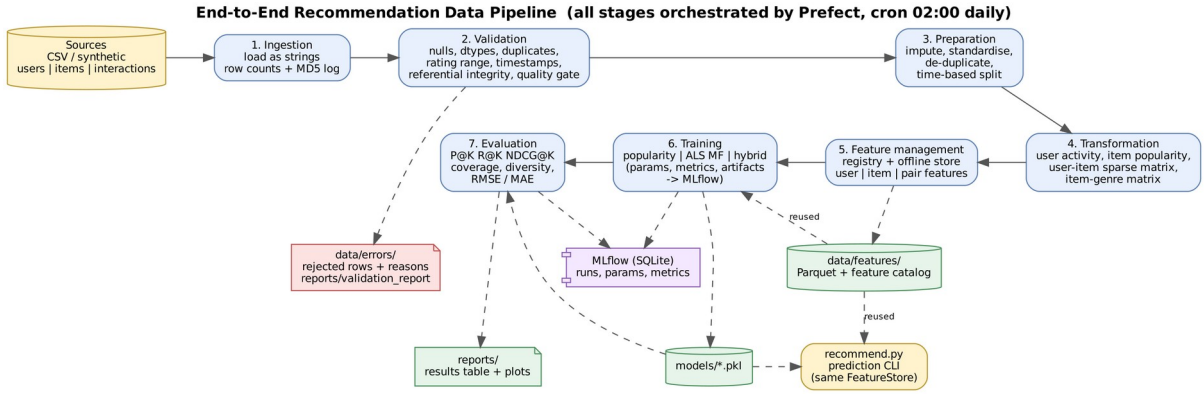


Figure 1: Architecture of the pipeline

Validation separates problems into errors and warnings. A row with an error, such as an invalid rating or an unknown user ID, is removed and saved in an error file together with the reason. A row with only a warning, such as a missing age, is kept and fixed later in preparation. I did this because removing a user for one bad attribute would also remove all of that user's interactions, which is worse for the model than filling in the age. The data is split by time and not randomly: the last 20% of the timeline is the test set, and features and models only use the training period, so there is no leakage from the future.

I used Python, Pandas and Parquet for the data, and my own rule-based validation code instead of Great Expectations so that each rule is easy to read and test. The feature layer is a small custom feature store on Parquet files, because Feast would be too heavy for a single machine. The models are written with NumPy and SciPy, experiments are tracked with MLflow, the workflow runs in Prefect, and the validation rules, split and metrics have unit tests in pytest.

3. Data validation and preparation

Table 1: Validation summary

Table	Records in	Valid	Rejected	Rejected %	Warnings
users	1,513	1,493	20	1.32	136
items	605	600	5	0.83	34
interactions	59,007	52,736	6,271	10.63	1,954

Most rejected interactions had ratings outside 1 to 5 (1,144), unreadable timestamps (874), user IDs not in the user table (1,030), item IDs not in the item table (732), exact duplicates (732), or future timestamps (477). The validation step also stops the pipeline if more than 15% of any table is rejected. I tested this by lowering the limit to 5%, and the validation task failed and none of the later tasks started. In preparation, if a user rated the same item more than once I kept the latest rating (883 rows removed), filled missing ages with the median, and standardised categories and countries. The clean data gives 41,482 training and 10,371 test interactions. From the training data I created a user activity summary, an item popularity table, an item-genre matrix and a sparse user-item matrix of size 1,493 by 599 (4.6% filled).

4. Feature management

The feature catalog (reports/feature_catalog.md) lists 23 user features, 10 item features and 3 user-item features with their definitions. Examples are the user's average rating, activity count, days since last interaction and preferred genre, and for items the popularity, average and smoothed rating, and age. All features are calculated as of the training cutoff. Training and the prediction script both read them through the same FeatureStore class, so the features used in training are the same as the ones used when recommending.

5. Models and results

I trained three models and logged their parameters, metrics and files in MLflow. The first is a popularity baseline that recommends the most popular items to everyone. The second is matrix factorisation with ALS, which gives personalised rankings and also predicts ratings. The third is a hybrid that combines the ALS score with a genre-preference feature and popularity; its weights were chosen on a validation part taken from the end of the training period. I evaluated on 1,345 users who had at least one test item rated 4 or higher, and items a user had already seen in training were never recommended.

Table 2: Results on the test set at K = 10

Model	Precision	Recall	NDCG	Coverage	Diversity	RMSE	MAE
Popularity	0.0502	0.1124	0.0833	0.055	0.850	0.895	0.731
ALS	0.0742	0.1721	0.1403	0.748	0.730	0.804	0.622
Hybrid	0.1016	0.2321	0.1956	0.758	0.233	n/a	n/a

Both personalised models are better than the baseline. ALS improves NDCG@10 by about 68% and lowers RMSE by about 10%, and the hybrid improves NDCG@10 by about 135%. The popularity model gives almost the same list to every user, so it covers only 5.5% of the items, while ALS covers about 75%. The hybrid is the most accurate, but its lists are much less varied (diversity 0.23) because the genre feature pushes users towards their main genres. The tuning chose a genre weight of 1.0 based only on NDCG; a weight of 0.5 would lose about 3.5% NDCG on the validation data but nearly double the diversity (0.44 against 0.24), so the best setting depends on what the platform wants. Results for K = 5 are in reports/evaluation_results.md.

6. Orchestration and reproducibility

The Prefect flow in dags/recommendation_flow.py runs the stages in order, and each stage starts only after the one before it has finished. A failed stage stops the flow. Transient stages are retried once, but the quality check is not retried because it would fail the same way again. A full run takes about 20 seconds, and the flow can be scheduled with a cron expression (daily at 02:00). Figure 2 shows a successful run.

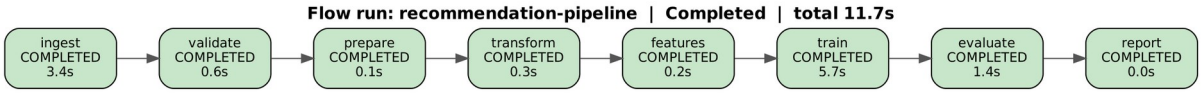


Figure 2: Successful run of the flow (status and time of each task)

7. Limitations

The data is synthetic, and the genre preference is built into the generator, so genre features are more useful here than they would be on real data. The gain of the hybrid over ALS is probably too optimistic and should be checked on a real dataset such as MovieLens. The results come from a single time split and one seed, so small differences should not be over-interpreted. Every test interaction involved a user and item seen in training, so cold start was not tested; the popularity model would be the fallback for new users. Finally, the ALS code is my own small NumPy version that works at this size but would need a library such as implicit for much larger data.